Factory discovery study sheet v2

October 1, 2026 · for Michael · v2, replaces v1 of Sep 30, 2026; every claim re-verified against Factory's own pages on Oct 1, 2026

The 26 engineering concepts behind Factory discovery, ordered by the value Factory delivers to the customer when the issue is present. For each: what it is, the value, how to check whether an account has it, how to size it, how Factory helps, and the caveats.

Read first

What a Droid does. You hand it a task — a ticket, an alert, a migration step. It reads the code, makes the change, runs the tests, and returns a pull request (PR: the proposal to merge a change) for a person to approve. It runs on an engineer's laptop, inside the build pipeline, or in Factory's cloud. It does investigation, and it does well-shaped repetitive change at scale. It does not approve, merge, or decide priorities, and it carries nothing that lives only in people's heads until someone writes it down (rule files, docs).

How each entry is built. What it is Value Precedent Check for it Size it Why it costs them How Factory helps Caveats. Every check can come back "no." When it does, skip the rest of the entry and move on: the account does not have that issue. Check before you size — a number on an issue the account does not have is worth nothing.

Labels. Every claim carries one. case — a customer story on Factory's own site, customer-reported through the vendor, no independent check. docs — Factory's own docs or announcements. inference — reasoning from those two, never a claim Factory makes. unknown — no source found. your rule — a rule Michael set. If you cannot label a number, drop it.

Precedent. A bonus tag, never the order. A — Factory did it for a named customer. B — Factory says it does it. C — a competitor's named customer did it and Factory has the surface. none — no precedent found. This version uses C nowhere: the competitor-case list from the Sep 28 thread is not recoverable, and nothing verified here earns the letter. Do not claim C from memory.

The order. Ranked by the value Factory delivers to the customer when the issue is present — compliance and downtime risk, recurring senior hours, and revenue timing, counted alike — judged from the evidence in each entry. It is a judgment call, not a measured figure, and it is yours to move. Ranks 19–26 are context: they size, price, or qualify the others rather than carry value themselves.

The order

Judged from the evidence in each entry: dated budget and forcing events first, recurring senior hours next, revenue timing alongside, context last. The one close call sits at 9 and 10 — entry 9 carries a published customer outcome; entry 10 gates every change but has Factory's surfaces only as evidence.

Rank	Concept	What's at stake	Precedent
1	**Legacy modernization and migrations**	The largest dated engineering budget; end-of-support risk	A (Comarch)
2	**Maintenance work: KTLO, tech debt, docs**	The largest recurring share of payroll	A (Chainguard), B
3	**Incidents and on-call**	Downtime revenue, senior hours, burnout	A (Empower), B
4	**The review wait**	Days every change sits idle; senior review hours	A (Empower, You.com), B

Rank	Concept	What's at stake	Precedent
5	**Feature speed and cost of delay**	Revenue timing on roadmap commitments	A (Nav, Groq)
6	**Cross-cutting changes across services and repos**	Hours multiplied by service count	A (Chainguard), B
7	**Security and compliance work**	Patch deadlines, audit evidence, penalties	B
8	**Triage**	Senior investigation hours on every incoming item	A (Empower), B
9	**Databases, APIs, data pipelines**	Dated upgrades; breakages with knowledge in heads	A (Comarch)
10	**Testing and coverage**	The speed limit on changing old code	B
11	**Codebase spread and orientation**	Reading hours before every change	A (Chainguard, Nav)
12	**Platform engineering backlog**	Cross-repo chores that never finish	B
13	**Unplanned work displacing the plan**	Roadmap dates; sizing input for 3, 5, 8	none
14	**Cloud migration leftovers**	Cloud bill; stalled changes in the unfinished part	none
15	**CI/CD**	Prerequisite for verifying agent work	B
16	**Environments, feature flags, rollback**	Release fear: freezes and batching	none
17	**Monolith**	Build tax; where modernization decisions sit	none
18	**Containers and Kubernetes**	Key-person risk; fits the deployment	B
19	**Enterprise adoption blockers**	Fit: where deals stall	B
20	**Engineering economics**	Converts hours to dollars; where budget sits	context
21	**Measuring engineering**	Proof: can a pilot produce a number	B
22	**The org: who buys, uses, blocks**	The economic buyer and the blockers	context
23	**Agents**	The product category; prerequisites	B
24	**Measured vs claimed**	Credibility: label every number	context
25	**Chat and in-editor assistants**	The baseline they compare to	context
26	**Autocomplete**	The baseline they compare to	context

Tier 1 — study first

1. Legacy modernization and migrations · A — Comarch

What it is Moving old code to a newer version, platform, or language. This is the largest and most dated engineering budget a company carries.

Value Company: the dated project lands with less outside help, and internal engineers stay on the roadmap; the risk of missing an end-of-support date comes down. VP Eng: the one project they are measured on lands on time.

Precedent A — Comarch (case, Jul 2026): an XSLT modernization Comarch estimated at 10,000 man-days was completed in 45 days; Comarch reports a 40% efficiency gain among its engineers and 30–34% higher team velocity, with projects that previously took over a year moving 18x faster.

Check for it Is there a modernization or migration with a date and a budget this year? What is it, from what to what, who is doing it, and what happens if the date slips?

Size it The budget (or outside rate × hours), the deadline, the engineers assigned, and the forcing event — end

of support, compliance, a cloud exit.

Why it costs them It is money paid to stand still: the budget line with the least upside and a hard date.

How Factory helps Droid Exec applies one change pattern across hundreds of files in parallel, headless, inside the pipeline (docs). Missions split large work into milestones and run it with worker agents and validators (docs). Comarch's XSLT project is the published migration case (case).

Caveats The Comarch numbers are the vendor quoting the customer's own estimates, with no independent check. COBOL and mainframe work is out — your rule (Sep 28), grounded in what Factory's Legacy Bench covers. A rewrite still needs human design decisions and review at every stage (inference).

2. Maintenance work: KTLO, tech debt, docs · A — Chainguard, B

What it is Keeping existing systems running — patches, upgrades, small fixes, support, and the documentation nobody writes. KTLO means keep the lights on.

Value Company: the payroll share spent on maintenance shrinks and goes back to the roadmap — the number a VP Eng most wants smaller. Engineer: less of the work nobody wants.

Precedent A — Chainguard (case, Jan 2026): 80 packages built across six repositories in one two-week session. B — Droid Exec is pitched for bulk refactors and bulk test updates (docs).

Check for it What share of engineering time goes to keeping things running versus building new? Which of that work repeats in the same shape?

Size it KTLO share × engineering payroll = the maintenance bill; hours per week on the repeated work.

Why it costs them It is headcount building nothing new, and it is the easiest work to put a number on.

How Factory helps Repeated, well-shaped maintenance — upgrades, patches, doc generation, bulk refactors — runs without supervision (docs; Chainguard published). Chainguard's stated mechanism: sessions that hold context for weeks, several in parallel, each in its own git worktree (case).

Caveats Value scales with how repetitive and how well-tested the work is (inference). Untested code needs tests first — see entry 10.

3. Incidents and on-call · A — Empower, B

What it is Production breaks; engineers on a rotation diagnose and fix. Diagnosis — finding which change and which code path caused it — is the slow part.

Value Company: downtime minutes and the revenue behind them, plus senior hours pulled off the roadmap. On-call engineer: fewer 2am investigations and less rotation fatigue, which shows up later as attrition.

Precedent A — Empower (case, Apr 2025): reduced incident response time by up to 40%. B — Incident Response: Droid watches a Slack alerts channel fed by Sentry, Rootly, Axiom, or Datadog, investigates, triages, prepares a fix, and keeps a runbook (docs, announced Jul 10, 2026). “Hundreds of hours saved in preview” is Factory’s own claim (docs).

Check for it How many incidents a month, and inside one, how long is diagnosis versus the fix? What does the diagnosis involve — logs, reading code, finding the change that caused it?

Size it Incidents per month × diagnosis hours × loaded rate; downtime cost per hour if the company has one.

Why it costs them Every incident stops a senior engineer mid-roadmap, and diagnosis is the part that reads code and logs — delegable work.

How Factory helps The agent starts the investigation when the alert fires: it reads the alert, the code, and the recent changes, then proposes a cause and a fix. The human decides and deploys (docs; Empower published).

Caveats Private Preview — access is requested, not generally available (docs). Needs the observability stack connected. The fix still goes through review. “Up to 40%” is hedged and has no published baseline; Empower's

page title drops the hedge — quote the body, not the title.

4. The review wait · A — Empower, You.com; B

What it is A change moves ticket branch PR review merge deploy. Most of that time is waiting, and the longest wait is usually for a reviewer.

Value Company: changes ship days sooner without adding reviewers. Senior engineer: fewer review hours per PR. Author: less time idle.

Precedent A — Empower (case, Apr 2025): automated code review, often reducing PR-created-to-approval times as much as 50%. You.com (case, Jun 2026): review "reliably catches security issues, bugs, and code-quality problems" before merge, rolling out as the default review setup. B — the Review Droid posts inline comments on PRs and merge requests on GitHub and GitLab, with optional security review (docs).

Check for it From a ticket being picked up to the change being live, where does the time sit? How long does a PR wait for first review, and how many approvers does it need?

Size it Days from PR open to approval; PRs per week per reviewer.

Why it costs them Senior time spent reading diffs, and every change idle while it waits — the wait usually outweighs the writing.

How Factory helps The Droid reviews first: it catches the mechanical issues and explains the change, so the human reviewer spends less time per PR and the queue moves (docs; Empower and You.com published).

Caveats Approvals stay with people; a Droid does not merge (inference — Factory makes no claim otherwise). It triggers on PR events and skips draft PRs, and automatic review is switched on at setup (docs). Fit rule: if review is the longest wait, producing code faster on its own lengthens the queue — the review product is the one that applies.

5. Feature speed and cost of delay · A — Nav, Groq, Comarch

What it is Product commits a roadmap and engineering delivers it. Every week a feature is late has a revenue or savings cost, and unplanned work displaces the plan.

Value Company: features reach customers sooner, and the deals and renewals tied to commitments stop slipping. EM: sprints stop being eaten by interrupts.

Precedent A — Nav (case, Feb 2025): 2x faster feature development cycles and a 60% reduction in context-switching time, with compliance requirements preserved. Groq (case, Dec 2025): 3x on medium-complexity tasks and 5x on quick-turn tasks — see the caveat. Comarch (case, Jul 2026): a customer-story implementation moved from 18 months to a 1-week MVP.

Check for it Which roadmap commitments are at risk this quarter, and who are they promised to? What share of a sprint goes to work that wasn't planned?

Size it The company's own cost-of-delay figure per week of slip; the deal or renewal value behind a commitment; the unplanned share of sprints.

Why it costs them Engineering capacity converts to revenue through the roadmap, and the delay has a dollar figure sales and product already know.

How Factory helps The agent takes the context-gathering and the small unplanned items so engineers stay on the feature. Nav's stated mechanism: context that took hours to assemble is now gathered up front (case). Groq engineers run several Droids in parallel on separate lines of work (case).

Caveats The multipliers are customer-reported through the vendor. Nav's 2x is an adoption claim with no published method. Groq's 3x and 5x are asterisked benchmark comparisons — Droid on Kimi K2/Groq versus GPT-5 on Codex and Claude Code, on groq/openbench — not before-and-after adoption figures; say which you are quoting. The gain is capped by the review wait (entry 4).

6. Cross-cutting changes across services and repos · A — Chainguard, B

What it is A change that must land in every service or repository — a library upgrade, a security patch, a framework migration. The cost multiplies by the service count.

Value Company: the long tail of a rollout finishes in days instead of quarters. Platform lead: the standardization they have owned for a year gets done.

Precedent A — Chainguard: six repositories in one session (case). Comarch runs Missions that coordinate work across dozens of agents (case). B — Droid Exec fans out across git worktrees to run the same change in parallel, and the docs carry batch and codemod patterns (docs).

Check for it How many services or repositories do you run, and when something has to change in all of them, how does it happen? How long did the last one take to reach the last service?

Size it Service count × hours per service on the last cross-cutting change; how many such changes a year.

Why it costs them The same change done N times by hand, by whoever owns each service — coordination plus repetition.

How Factory helps One defined change applied across many repos in parallel, each landing as its own PR for that repo's owner to review (docs; Chainguard published).

Caveats Each repo's tests and pipeline decide how safely it lands. A service with no owner still needs a person to approve it.

Tier 2

7. Security and compliance work · B

What it is Mandatory, dated work — patching vulnerabilities (known weaknesses in their code or the libraries it uses), updating dependencies, producing audit evidence — plus the change controls that govern every release in regulated industries.

Value Company: patch deadlines met without pulling roadmap engineers, and an audit trail on every change. Security lead: a backlog that drops instead of growing.

Precedent B — optional security review on every PR, methodology STRIDE, OWASP, OWASP LLM Top 10, and supply chain (docs). SOC 2 Type II, ISO 27001, ISO 42001, an isolated EU deployment, OpenTelemetry export, and fully airgapped deployment (docs; trust center). SSO/SCIM starts at the Business plan (docs). Nav kept its compliance requirements intact (case).

Check for it How many open vulnerabilities, and what's the patch deadline? Who does the patching, and what does it displace?

Size it Open vulnerabilities × hours each; audits per year × engineer-days gathering evidence.

Why it costs them Work with a penalty behind it and no product upside, done by senior people at the worst time.

How Factory helps Dependency and patch changes are bulk, well-shaped work for Droid Exec. A security check runs on each PR. The enterprise controls are what get the tool itself through their security review (docs).

Caveats Their security review of the tool gates the deal — find its owner on the first call (inference). On GDPR: Factory shows a compliance badge and GDPR-style privacy provisions, but docs claim no certification, and none exists — don't repeat one. Regulated change control still applies to every agent PR.

8. Triage · A — Empower, B

What it is The decisions on every incoming item — real? how bad? whose? when? — which land on senior engineers because the judgment needs system knowledge.

Value Company: senior hours returned, and faster answers to customers. Senior engineer: the interrupt queue stops eating their week.

Precedent B — Incident Response triages alerts and filters noise (docs), and the Automations templates include Triage (docs). A — Empower (case, Apr 2025): reduced delays between product and development Q&A by up to 50%.

Check for it Who does first-pass triage, and how many items a week reach them? How long before someone knows whether an item is real and where it is in the code?

Size it Items per week × minutes each × loaded rate; the investigation share of each item.

Why it costs them The investigation half — reproduce, read logs, find the code path — is expensive senior time; the decision half is cheap.

How Factory helps The agent does the investigation and hands the senior a reproduced, located issue to decide on (docs for alerts; inference for tickets and product questions).

Caveats The decision stays human. Value depends on how much of each item is investigation rather than judgment.

9. Databases, APIs, data pipelines · A — Comarch

What it is The contracts between systems. An API is how one program calls another; a database holds the product's data; a pipeline moves data between systems. Changing one means changing everything that depends on it.

Value Company: dated upgrades (end-of-support databases) done on time, and pipeline breaks fixed without a scramble. Data or platform owner: fewer early-morning breakages.

Precedent A — Comarch (case, Jul 2026): a database migration scoped to take 45 days was completed by a Factory mission in 6 hours; the scope is Comarch's own estimate. B — bulk change surfaces for the mechanical part (docs).

Check for it Which databases are you on, and are any near end of support? How many APIs do customers or other teams depend on, and how many versions do you keep supported?

Size it Engines and versions with dates; API versions supported; pipeline breaks per month × hours each.

Why it costs them Forced work with a date, where the knowledge sits in people's heads.

How Factory helps The mechanical part of a version or API migration across every caller is repeatable work, and breakage investigation is the triage pattern from entry 8 (inference from docs surfaces; Comarch published for the database migration).

Caveats Data migrations carry irreversible risk — human sign-off at each stage (inference). No published outcome exists for API version migrations — unknown; the Comarch figure covers one database migration, quoted by the vendor.

10. Testing and coverage · B

What it is Tests are code that checks code; coverage is how much of the code they check. Coverage sets how fast a team can change code safely.

Value Company: changes to old code stop waiting on manual confidence, and the migration nobody dared start becomes startable. Engineer: a red light they can trust.

Precedent B — Droid Exec for generating and updating tests in bulk (docs).

Check for it On the parts you most need to change this year, how well tested are they? What has been put off because of it?

Size it Coverage on the parts that matter; manual QA hours per release; the decision waiting on confidence.

Why it costs them Without a fast check, every change waits on slow human confidence — and an agent's changes cannot be verified cheaply either.

How Factory helps Writing tests for untested code is well-shaped bulk work, and it unlocks the agent's own speed on that code afterward (docs surface; the unlock is inference).

Caveats Agent-written tests still need review for whether they test the right thing. “Percent of tests automated” is not coverage — ask for coverage. This entry gates everything, including the agent; it sits below entry 9 only because its published evidence is thinner. Move it up if you weight the mechanism over the evidence.

11. Codebase spread and orientation · A — Chainguard, Nav

What it is A repo is the code plus its history; the shared copy on GitHub or GitLab is where every change lands. The cost is reading before writing, multiplied by how many repos one change touches.

Value Company: the hours spent getting oriented per change come back. Engineer: a task starts with the context already gathered.

Precedent A — Chainguard: one session reading across six repositories and upstream sources (case). Nav: context that took hours to assemble now gathered up front, with 100% of engineering context automatically unified (case).

Check for it Does one change usually touch more than one repo? How long does it take to get oriented before anyone writes a line?

Size it Repos per change; orientation hours per change; changes per month.

Why it costs them Reading time before writing, on every change, by senior people.

How Factory helps The agent does the orientation across repos and holds it across a long session — Chainguard's stated mechanism is compaction (case; docs).

Caveats Self-hosted GitHub Enterprise Server or GitLab connects only on the Enterprise plan (docs) — ask where the code is hosted on the first call.

12. Platform engineering backlog · B

What it is The internal team that builds the paved road — pipeline templates, deploy tooling, a developer portal — whose backlog is standardization across every repo.

Value Company: teams moved onto the standard path, which makes every later rollout cheaper. Platform lead: a multi-repo chore list that finishes.

Precedent B — Droid Exec runs headless in CI/CD and across repos (docs); Automations run Droid on schedules, GitHub events, and webhooks (docs).

Check for it Is there a platform team, and what's on their backlog that touches every team? How long has it been there?

Size it Repos off the standard path × hours to move each.

Why it costs them A small team with a cross-repo chore list that never ends.

How Factory helps Multi-repo standardization is the fan-out pattern from entry 6 (docs). The platform team is also a buyer in its own right, with a productivity metric to move (inference).

Caveats Adoption of the standard path is a people problem the agent doesn't fix.

Tier 3

13. Unplanned work displacing the plan · none

What it is A sprint is fixed capacity. Bugs, escalations, and deal requests enter it and push planned work out one for one.

Value Company: roadmap dates hold. EM: a sprint that ends where it was planned.

Precedent None specific; the mechanism is carried by entries 3, 5, and 8.

Check for it What share of a sprint ends up on work that wasn't planned at the start? Where does it come from?

Size it Unplanned share × team capacity; carry-over per sprint.

Why it costs them Delayed features, and a velocity that reads flat with stable headcount.

How Factory helps The small unplanned items — bugs, investigations, quick questions — are what the agent absorbs (inference from the incident and triage patterns).

Caveats A sizing input, not a lever on its own. Size it here; sell on entries 3, 5, and 8.

14. Cloud migration leftovers · none

What it is What is still on-prem, or was lifted into the cloud unchanged in the 2015–2020 move; whether infrastructure is defined in code (Terraform and the like).

Value Company: the cloud bill, and the stalled changes in the unfinished part.

Precedent None Factory-published; infrastructure-as-code changes are code changes (inference).

Check for it What's still in the state it was moved in? Is your infrastructure defined in code, and who changes it?

Size it Cloud bill trend; stalled changes in the leftovers.

Why it costs them A CFO-visible bill plus a modernization backlog.

How Factory helps Infrastructure code can be delegated like any other code (inference).

Caveats No Factory precedent. Treat it as entry 1 when the work is code, as context when it isn't.

15. CI/CD · B

What it is The automation from commit to production — build, test, deploy. Its speed and reliability cap how fast anything ships.

Value Company: the pipeline is where an agent's work gets verified — a prerequisite more than a gain.

Precedent B — Droid Exec runs in GitHub Actions, GitLab CI, and on self-hosted runners you control (docs).

Check for it Does every PR run through CI before merge, and how long does the pipeline take? How often does it fail for reasons unrelated to the change?

Size it Pipeline minutes × runs per day; the unrelated-failure rate.

Why it costs them A long or flaky pipeline is paid on every PR, and it makes engineers batch changes, which makes reviews worse.

How Factory helps The agent runs inside the pipeline and uses it to check its own work (docs).

Caveats No CI means no cheap verification of agent output — weak fit until it exists (inference).

16. Environments, feature flags, rollback · none

What it is Code runs in stages — dev, staging, prod. A feature flag ships code switched off and turns it on later; a rollback returns production to the previous version.

Value Company: less fear per release, so fewer freezes and release trains.

Precedent None.

Check for it How well does staging match production, and how do you get back when a release goes wrong? When did that last happen?

Size it Rollback time; release frequency; freeze periods per year.

Why it costs them Fear of release turns into batching and waiting.

How Factory helps Indirectly: more, smaller agent-made changes need flags and a fast rollback to be safe (inference).

Caveats A sizing concept for release risk, not a lever.

17. Monolith · none

What it is One application, one deploy, one codebase — the default until size and team count make one deploy risky and one codebase contended.

Value Company: where changes are slowest, and where the modernization decision sits.

Precedent None specific.

Check for it Is the core one application, and how long does a full build and test run take? How many teams ship into the same deploy?

Size it Build and test minutes; teams per deploy.

Why it costs them Every change pays the build tax, and coordination grows with every team added.

How Factory helps Whole-codebase context matters most here — the Chainguard and Nav mechanism (inference). Breaking a monolith into modules is bulk work (inference).

Caveats A context concept — sell on entries 1, 4, and 11.

18. Containers and Kubernetes · B

What it is A container packages an application so it runs the same anywhere; Kubernetes runs and scales many of them. The operating burden lands on a platform or SRE team.

Value Company: key-person risk — the two people who understand the cluster.

Precedent B — Droid runs in their environment as-is: the engineer's machine, self-hosted CI runners, Droid Computers (including bring-your-own-machine), and fully airgapped deployment (docs).

Check for it What share of services run in containers, and how many people could fix the cluster at 2am? What are they not doing because they're that person?

Size it Non-containerized share; number of cluster experts.

Why it costs them A specialist in every deploy path, and a risk with a name on it.

How Factory helps It runs where the code runs (docs). Moving services into containers is code work (inference).

Caveats Operating knowledge isn't replaced by an agent. A context concept.

Context and fit

These eight carry little value on their own. They decide whether a deal can happen, convert the entries above into dollars, or set the baseline the buyer compares Factory to. Read them before calls; sell on the entries above.

19. Enterprise adoption blockers · B

What it is The four places agent adoption stalls in a large org: the security review of the tool, codebase context the agent can't see, knowledge that lives only in heads, and the review bottleneck moving downstream when more PRs land on the same reviewers.

Value For you — the fit test. Each stall either has a product answer or it doesn't.

Precedent B — SOC 2 Type II, ISO 27001, ISO 42001; airgapped and isolated-EU deployment; SSO/SCIM from

the Business plan up; self-managed GitHub Enterprise Server and GitLab on the Enterprise plan (docs; trust center).

Check for it What deployment models are allowed for AI tools here — hosted, in your cloud, on-prem? Who reviews PRs today, and how many each a week?

Size it How long the last security review of a new tool took; review capacity per reviewer; the state of documentation and rule files.

Why it costs them Each blocker is a place the deal stalls, and a pilot that ignores them produces nothing.

How Factory helps Deployment options and certifications answer the security stall. The Review Droid answers the review stall. Context is rebuilt from the repo and docs (docs).

Caveats Knowledge in heads has to be written down first — rule files, docs — which is a setup cost on the customer's side (inference).

20. Engineering economics · context

What it is A fully-loaded engineer costs salary plus benefits, taxes, equipment, and space; headcount is the budget; finance books engineering pay as expense or as a capitalized asset.

Value The conversion: every hour in the entries above becomes dollars here, and it tells you where tooling budget sits.

Precedent Context. Public pricing (company-disclosed, verified Oct 1, 2026): Teams $60/month per team plus $40 per seat, capped at 10 seats; Business and Enterprise custom, with SSO/SCIM from Business. Individuals: Pro $20, Plus $100, Max $200 per month.

Check for it What loaded cost do you use when you make a case, and how is headcount planned for next year? Where does developer tooling budget sit, and who signs?

Size it Loaded hourly rate; open reqs; the tooling line item.

Why it costs them Flat headcount against a growing roadmap is the pressure that funds tools; unfilled reqs are a cost line a tool can stand in for.

How Factory helps The comparison is loaded hours returned against seat cost (inference). Mind the 10-seat cap on Teams: above it the deal is Business at custom pricing, and the seat math changes.

Caveats None — context only.

21. Measuring engineering · B

What it is DORA's four metrics (deployment frequency, lead time, change failure rate, time to restore), SPACE, DevEx surveys — and what the VP Eng reports upward.

Value The proof — whether a pilot can produce a before-and-after number.

Precedent B — OpenTelemetry export of Droid activity by default, a hosted analytics API, and an agent-effectiveness surface connecting spend to output (docs).

Check for it What do you measure engineering on, and where does it live? Which number would you most like to move this year?

Size it The metrics that exist; the one number they'd pay to move.

Why it costs them No dashboard means building one during the pilot, and a gameable metric shapes how the tool gets judged.

How Factory helps Agree the metric the pilot must move before it starts (inference). Droid usage exports to their observability stack (docs).

Caveats Vendor multipliers are never their number.

22. The org: who buys, uses, blocks · context

What it is Engineers by level, an engineering manager (EM) per team, directors, a VP of Engineering, a CTO; around them product managers, architects, SRE, security, QA.

Value For you — the economic buyer and the blockers.

Precedent Context.

Check for it Who owns the engineering budget, and what number are they held to? Who had concerns the last time a new tool came in?

Size it Team count; EM span; prior AI tooling decisions and who made them.

Why it costs them Selling below the budget owner adds a stage to every deal.

How Factory helps Context only.

Caveats None.

23. Agents · B

What it is Delegation instead of assistance. The agent plans, reads, changes, tests, and returns a PR; it runs at an autonomy level from ask-first to run-to-completion; it is steered by rule files in the repository (AGENTS.md) and can be wired to tickets, CI, monitoring, and docs.

Value The product category — the sum of the entries above.

Precedent B — Droid CLI with autonomy levels (Off, Low, Medium, High), custom droids (subagents with their own prompt, model, and tool policy), Spec Mode (plans before editing, waits for approval), and Missions (docs).

Check for it Have you run a coding agent end to end on a real task? What would an agent find in the repo to work from — rules, tests, a pipeline?

Size it Prior trials and how they were set up; which prerequisites exist today.

Why it costs them A trial run without rule files, tests, or a pipeline says nothing about fit.

How Factory helps Spec Mode writes the plan and waits for approval before touching files; autonomy levels gate what runs unattended (docs).

Caveats The prerequisites decide the result.

24. Measured vs claimed · context

What it is Four kinds of evidence — vendor studies, randomized trials, DORA survey findings, benchmarks — each with a known weakness.

Value Credibility: the seller who labels every number earns the right to be believed.

Precedent Factory publishes benchmark results (docs), and who runs each one matters. Legacy Bench is Factory's own (vendor-run; Droid 42.5%, first, Apr 2026). Terminal Bench is third-party from tbench.ai (Droid 63.1%, first, Dec 2025). The Review Benchmark is open-source (13 models, 50 PRs, 167 validated bugs; Apr 2026). Agent Arena is crowd-voted by users (Droid first at 1330 Elo; Dec 2025).

Check for it What have you seen or run that measured AI tooling's effect? What would count as a pilot working?

Size it Their prior evidence; the pilot's done-check.

Why it costs them Buyers have seen claims that didn't show up in their own metrics.

How Factory helps Context only. Label each benchmark by who runs it.

Caveats Say so.

25. Chat and in-editor assistants · context

What it is ChatGPT and Cursor-type tools: a person directs each step and reviews each edit, and produces more

code faster into the same review queue.

Value The baseline the buyer compares Factory to.

Precedent Context.

Check for it What AI tools do engineers use day to day, and which are approved? What did more code output do to reviews?

Size it Tools in use, sanctioned or not; the change in review load.

Why it costs them Shadow use is a security exposure and a sign of demand; a bigger review queue is where the pain moved.

How Factory helps The difference is delegation — work leaves the engineer's calendar — rather than faster typing (inference).

Caveats Context only.

26. Autocomplete · context

What it is GitHub Copilot and similar: it suggests the next lines as an engineer types. It speeds the typing segment and nothing else.

Value The baseline. If they saw nothing from Copilot, they'll expect nothing from the next tool.

Precedent Context.

Check for it Which assistants are in use, and what did you see change? What share of a change's total time is the writing part?

Size it Seats; what they measured.

Why it costs them The effect is capped by how much of cycle time was ever typing — usually the smallest segment.

How Factory helps It works on the waiting segments autocomplete never touched: diagnosis, review, bulk change (inference).

Caveats Context only.

Sources

All Factory pages fetched October 1, 2026. Every customer figure is customer-reported through the vendor, with no independent check.

Factory-published cases. Chainguard (Jan 30, 2026) — factory.com/case-studies/chainguard: 80 packages, 6 repositories, one two-week session; compaction as the stated mechanism. Empower (Apr 7, 2025) — factory.com/case-studies/empower: up to 40% incident response, as much as 50% PR-to-approval, up to 50% product–development Q&A. Nav (Feb 25, 2025) — factory.com/case-studies/nav: 2x feature cycles, 60% context-switching time, compliance preserved. Groq (Dec 30, 2025) — factory.com/case-studies/groq: 3x/5x benchmark comparisons, asterisked on the page. Comarch (Jul 29, 2026) — factory.com/case-studies/comarch: XSLT 10,000 man-days 45 days; database migration 45 days 6 hours; 40% efficiency; 30–34% velocity; 18x on year-plus projects. You.com (Jun 16, 2026) — factory.com/case-studies/you-com: qualitative only — review as default setup, a debug from days to an afternoon, Missions opening PRs, agents running 24/7.

Announcements. Factory 2.0 (Jun 15, 2026) — factory.ai/news/software-factory: customer list NVIDIA, EY, Adobe, Palo Alto Networks, Adyen, Blackstone, Wipro, Comarch. Incident Response (Jul 10, 2026) — factory.ai/news/incident-response: Sentry, Rootly, Axiom, Datadog; runbook; Private Preview per docs; “hundreds of hours saved in preview” is Factory's own claim.

Factory-disclosed (docs.factory.com). Droid Exec (headless, worktree fan-out, batch and codemod patterns, GitHub Actions, GitLab, self-hosted runners). Missions (1–500 features; workers and validators). Automated code review (GitHub and GitLab; triggers on PR events; skips drafts; optional security review: STRIDE, OWASP, OWASP LLM Top 10, supply chain). Spec Mode and Interaction Modes; autonomy levels Off, Low, Medium, High; AGENTS.md; custom droids (subagents). Droid Computers (bring-your-own-machine and managed). Automations (schedule, Slack, GitHub events, webhooks; templates include Triage, QA, Security Audit, AutoWiki, Incident Response). Self-managed GitHub Enterprise Server and GitLab — Enterprise plan only. Airgapped deployment; isolated EU deployment. SOC 2 Type II, ISO 27001, ISO 42001 (trust.factory.com). OpenTelemetry export by default. Pricing: Teams $60 per team per month plus $40 per seat, up to 10 seats; Business and Enterprise custom; Pro $20, Plus $100, Max $200 for individuals. Benchmarks: Legacy Bench (Factory's own, Apr 2026); Terminal Bench (tbench.ai, Dec 2025); Review Benchmark (open-source, Apr 2026); Agent Arena (crowd-voted, Dec 2025); NextJS Evals also published.

Carried from v1, not re-verified this pass. Bird, Nagappan, Murphy, Gall, Devanbu — “Don't Touch My Code!”, ESEC/FSE 2011 — code ownership and defects.

Your rules. COBOL and mainframe work excluded (Sep 28, 2026), grounded in what Factory's Legacy Bench covers. Precedent letters A/B/C (Sep 28). Check before size (Sep 30).